

Library Borrowing

A step-by-step object-oriented design walkthrough

CSC241 - Object Oriented Programming (Java)

Lecture 08 - BCS

Dr. Muhammad Shahid Bhatti

Assistant Professor, Department of Computer Science

COMSATS University Islamabad, Lahore Campus

1. Read the scenario

A university library wants a simple system for borrowing books. Each member has a unique member ID. Each book has a title, and the library may have several physical copies of the same book. Every physical copy has its own copy ID and availability.

A member requests to borrow a particular book copy. For this exercise, a member may have **one current loan at a time**, and a physical copy may have **one active borrower at a time**.

The system checks whether the member already has a current loan and whether the requested copy is available. If both checks allow borrowing, the system accepts the request. It creates a loan record containing the member, copy, borrowing date, due date and an ACTIVE status. The copy becomes unavailable, and the member remembers the new loan.

If either check fails, the system rejects the request. It creates no new loan and leaves the existing information unchanged.

Example: M01 requests COPY-01 of Java Basics. M01 has no current loan and the copy is available, so borrowing succeeds. If M02 then requests COPY-01, that request fails because the copy is already borrowed.

Scope of this exercise

Borrowing and rejection are included. Returning books and loan history are later extensions. Dates are supplied as strings. The domain implementation uses ordinary fields and object references, with no arrays or ArrayList. All identifiers are fictional teaching data.

The walkthrough follows: scenario, nouns and verbs, class decisions, responsibilities, diagrams, collaboration, state changes, source folders and complete Java code.

2. Identify nouns, verbs and candidate classes

Nouns suggest concepts or information. Verbs suggest actions. These candidates still need design decisions.

Nouns and noun phrases	Verbs and actions
Member, member ID	Requests borrowing
Book, title, physical book copy	Checks whether a loan already exists
Copy ID, availability	Checks availability; accepts or rejects
Loan record, dates, status	Creates a loan; marks copy unavailable
Borrowing request	Records the loan for the member

Which nouns become classes?

Candidate	Decision	Reason
Member	Class	Identity and a current loan
Book	Class	Title-level information
BookCopy	Class	One physical copy with its own state
Loan	Class	A relationship with dates and status
Member ID, title, copy ID	Fields	Information owned by another object
Availability, dates, status	Fields	State of BookCopy or Loan
Borrowing request	Use-case operation	No separate request class needed here

Why Book and BookCopy? Two copies can share the same title, while one is available and the other is borrowed. Availability therefore belongs to BookCopy.

Where does LibraryDesk come from? We introduce it as a coordinator. It is a design choice, rather than a noun explicitly stated in the scenario.

A verb can describe an action, such as checking availability, or an association, such as a copy belonging to a book. The next step assigns ownership.

3. Assign responsibilities and collaborators

CRC means **Class, Responsibilities, Collaborators**. A responsibility describes what an object knows or does. A collaborator is another object it needs to perform its work.

Class	Responsibilities	Collaborators
Member	Know its ID. Know whether it has a loan. Remember its current loan.	Loan
Book	Know and provide title-level information.	None for title lookup
BookCopy	Know its copy ID, Book and availability. Accept borrowing only when available. Become unavailable after acceptance.	Book
Loan	Connect one Member with one BookCopy. Record dates and status.	Member, BookCopy
LibraryDesk	Coordinate the request. Check the member, ask the copy, create a Loan and record it for the member.	Member, BookCopy, Loan

Ownership decisions

BookCopy controls availability. Its availability field is private. The coordinator asks `tryBorrow()` to enforce the rule at the object boundary.

Member remembers its current loan. In this bounded example, `LibraryDesk` checks `hasLoan()` before using `setLoan(loan)`. Borrowing requests follow this coordinator path.

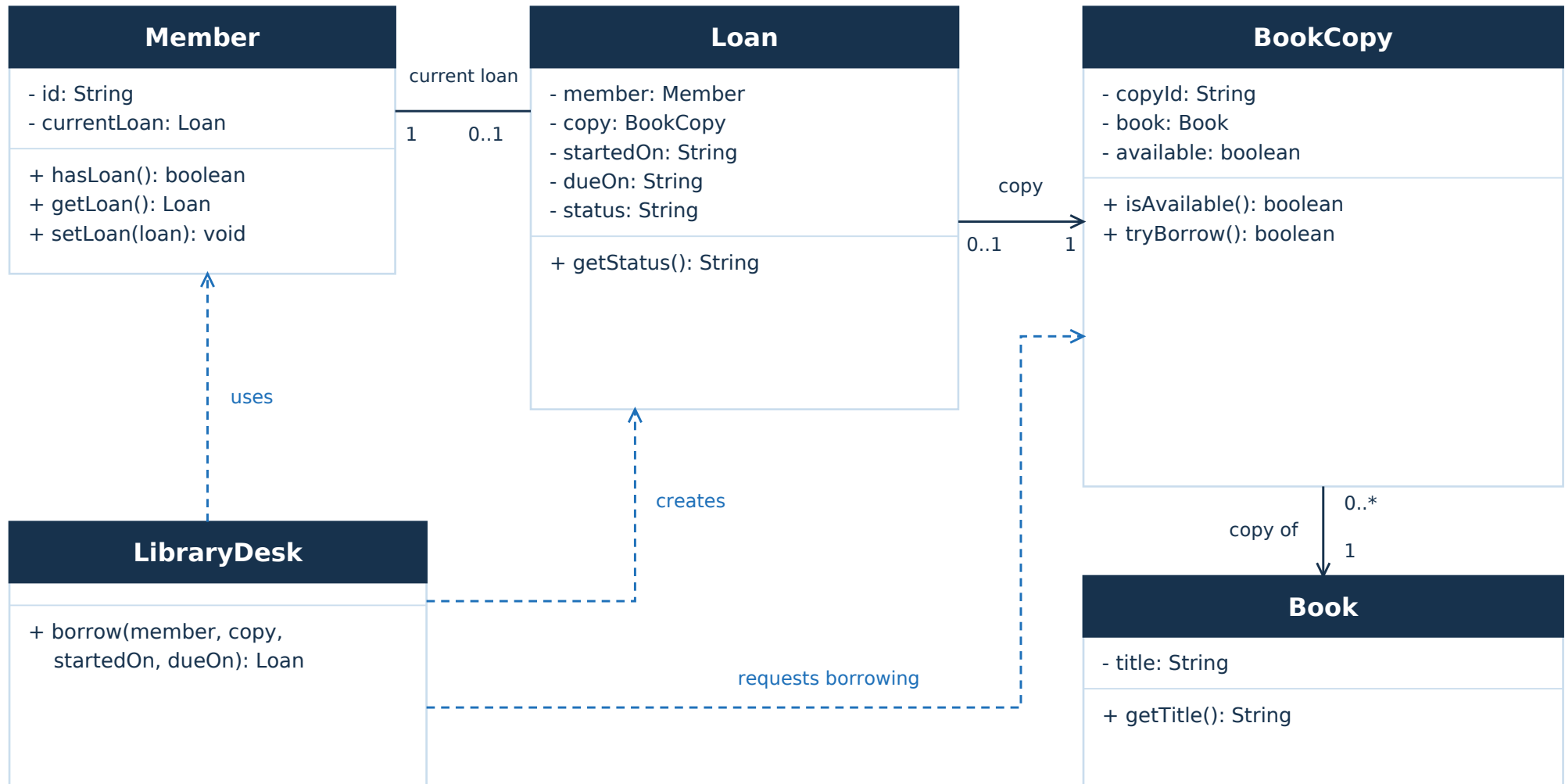
Loan holds the relationship data. The member and copy fields refer to existing objects; they do not create duplicate objects.

LibraryDesk coordinates. It uses the supplied objects during a request and does not keep them in persistent fields.

`BookCopy` refers to `Book` for title information. Its availability check uses its own boolean field and does not need to ask `Book`.

4. UML class diagram

Key fields and operations are shown. Constructors and some getters are omitted from the diagram; the code appendix includes them.



Solid lines show associations. A solid arrow shows navigation through a reference. Blue dashed arrows show dependencies. Multiplicities describe current loans in this exercise.

5. Read the associations and reference fields

Association	Meaning
Member - Loan	A Member has zero or one current Loan. Every Loan belongs to exactly one Member.
Loan - BookCopy	Every Loan refers to exactly one BookCopy. A BookCopy participates in zero or one current Loan.
BookCopy - Book	Every BookCopy refers to exactly one Book. A Book can have zero or many physical copies.

Notation	Meaning
1	Exactly one
0..1	Zero or one
0..*	Zero or many
- before a field	Private
+ before an operation	Public
Solid line / arrow	Association; arrow adds a navigation direction
Dashed arrow	Dependency: one class uses another

How the associations appear in Java

Member has a `currentLoan` field of type `Loan`. `Loan` has `member` and `copy` fields of types `Member` and `BookCopy`. `BookCopy` has a `book` field of type `Book`. These are object references, not collections.

`LibraryDesk` receives `Member` and `BookCopy` as parameters and constructs a `Loan`. These are dependencies because it uses the objects during the operation without storing them as fields.

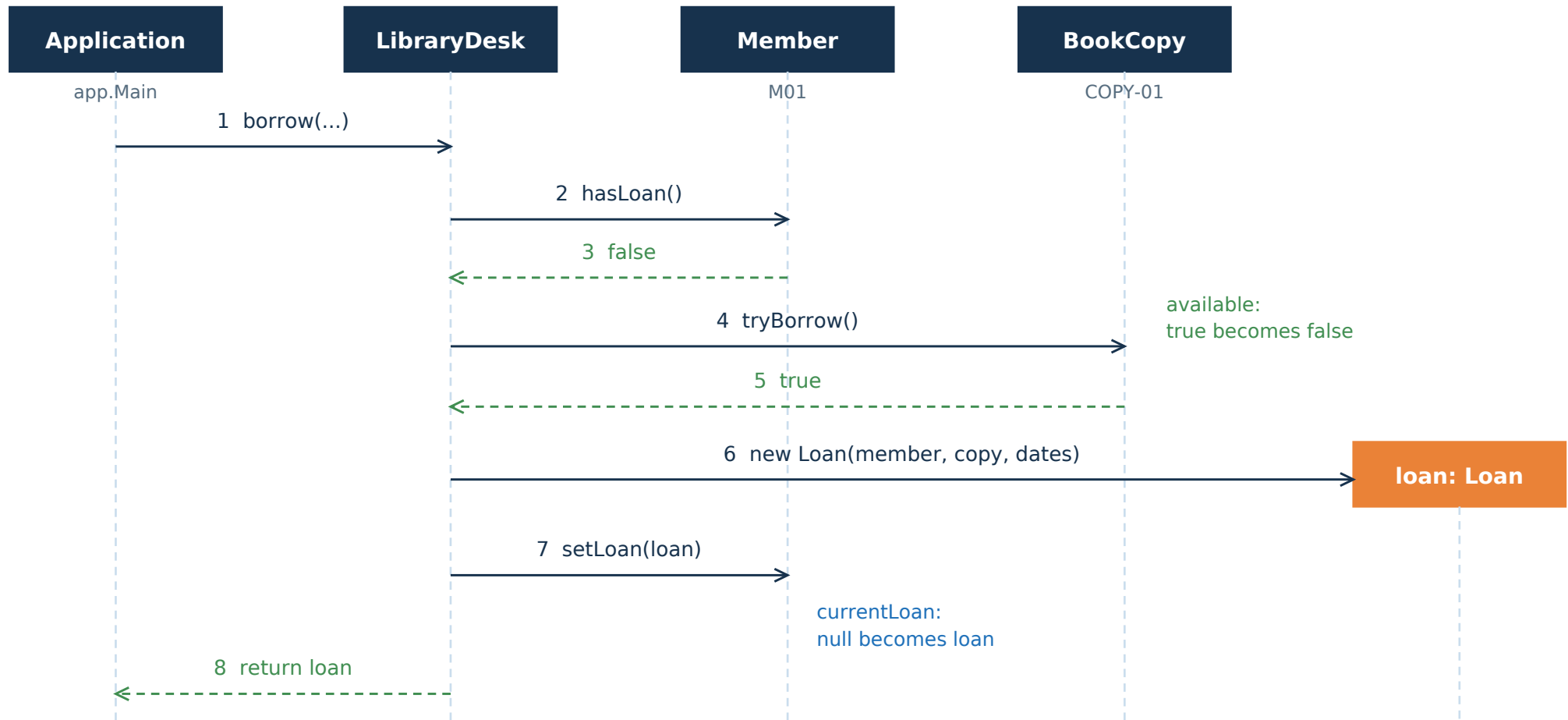
Current loans versus history

This diagram describes current loans only. If we later store history, a member and a copy could each be associated with many historical `Loan` records. The rule that a copy can have only one active borrower would still apply.

No ownership or destruction rule is specified that requires an aggregation or composition diamond. Ordinary associations express the relationships needed here.

6. Sequence diagram: a successful request

Time runs downward. Application starts the use case. M01 has no loan, and COPY-01 is available.



Solid arrows are calls or object creation. Dashed arrows are results returned to the caller. The Loan lifeline begins at creation. The diagram covers success; rejection paths are described next.

7. Trace the complete borrowing request

Initial state: M01 has no current loan. COPY-01 is available. The supplied borrowing date is 2026-10-03 and the due date is 2026-10-10.

1. The application sends the request.

It calls `LibraryDesk.borrow(member, copy, startedOn, dueOn)`, supplying the objects and date strings.

2. LibraryDesk checks the member.

It calls `member.hasLoan()`. The result is false, so the request can continue.

3. LibraryDesk asks the copy to accept borrowing.

It calls `copy.tryBorrow()`. The copy is responsible for enforcing availability.

4. BookCopy checks and updates its state.

Because `available` is true, it changes `available` to false and returns true.

5. LibraryDesk creates a Loan.

The constructor stores references to M01 and COPY-01, records the supplied dates and sets status to ACTIVE.

6. LibraryDesk records the loan for the member.

It calls `member.setLoan(loan)`. The `currentLoan` field changes from null to a reference to the new Loan.

7. LibraryDesk returns the successful Loan.

The application uses the result to display success and the loan details.

What the objects accomplished together

BookCopy protected its availability rule. Loan recorded the successful relationship. Member remembered the record. LibraryDesk arranged their work in order.

The numbered workflow above groups the eight diagram messages into seven activities: the `hasLoan()` call and its false response form one activity.

8. Compare state and trace rejected requests

State	Before M01 request	After success
COPY-01 available	true	false
M01 currentLoan	null	Reference to new Loan
New Loan object	Does not exist	Exists, status ACTIVE
Member ID	M01	M01, unchanged
Copy ID	COPY-01	COPY-01, unchanged
Book title	Java Basics	Java Basics, unchanged
Loan dates	No new record	03 Oct to 10 Oct 2026

null means a reference field currently refers to no object. After success, currentLoan refers to the same Loan object returned by the coordinator.

Rejection 1: the member already has a loan

LibraryDesk sees member.hasLoan() return true and returns null immediately. It does not call tryBorrow(), so the requested copy stays in its original state. The existing loan stays unchanged.

Rejection 2: the copy is unavailable

BookCopy.tryBorrow() returns false without changing availability. LibraryDesk returns null without creating a Loan or updating the member.

Request	Result and unchanged information
M02 requests borrowed COPY-01	Reject. COPY-01 stays unavailable. M02 currentLoan stays null. M01 loan stays unchanged.
M01 requests available COPY-02	Reject because M01 already has a loan. COPY-02 stays available. M01 keeps its existing loan.

The small implementation assumes valid, non-null inputs and one request at a time. Date validation, returning, history and simultaneous requests are outside this exercise.

9. Organize, compile and check the program

Use a project folder named **03_LibraryBorrowing**. Put the four domain classes in package model, and LibraryDesk and Main in package app.

Folder	Source files	First line
src/model	Member.java, Book.java, BookCopy.java, Loan.java	package model;
src/app	LibraryDesk.java, Main.java	package app;
out/model	Generated domain .class files	Compiler output
out/app	Generated application .class files	Compiler output

Open Windows CMD in 03_LibraryBorrowing, the folder containing src. Create out once, then compile and run:

```
mkdir out
javac -d out src\model\*.java src\app\*.java
java -cp out app.Main
```

Keep .java files in src. The compiler generates package folders and .class files inside out. Use a JDK. No external Java libraries are required.

Output checkpoints

Checkpoint	Expected output
Before M01 borrows	Available: true; Current loan: null
Successful request	Request: SUCCESS; Due date: 2026-10-10
After success	Available: false; Current loan: ACTIVE
M02 requests the copy	Second member rejected: true; Current loan: null
M01 requests another copy	Existing borrower rejected: true; COPY-02 remains available: true

Questions to answer before copying the code

Why are Book and BookCopy separate? Which object owns availability? Why is Loan a class? Which fields change after success? Why does COPY-02 remain available after M01 is rejected?

The complete code follows. String[] args is the supplied Java entry-point signature; this exercise does not access or manipulate that array.

10. Complete Java code: Member and Book

src/model/Member.java

```
package model;

public class Member {
    private final String id;
    private Loan currentLoan;

    public Member(String id) {
        this.id = id;
        this.currentLoan = null;
    }

    public String getId() {
        return id;
    }

    public boolean hasLoan() {
        return currentLoan != null;
    }

    public Loan getLoan() {
        return currentLoan;
    }

    public void setLoan(Loan loan) {
        this.currentLoan = loan;
    }
}
```

src/model/Book.java

```
package model;

public class Book {
    private final String title;

    public Book(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }
}
```

10. Complete Java code: BookCopy

File: src/model/BookCopy.java. tryBorrow() owns the availability check and changes state only on acceptance.

```
package model;

public class BookCopy {
    private final String copyId;
    private final Book book;
    private boolean available;

    public BookCopy(String copyId, Book book) {
        this.copyId = copyId;
        this.book = book;
        this.available = true;
    }

    public String getCopyId() {
        return copyId;
    }

    public Book getBook() {
        return book;
    }

    public boolean isAvailable() {
        return available;
    }

    public boolean tryBorrow() {
        if (!available) {
            return false;
        }

        available = false;
        return true;
    }
}
```

10. Complete Java code: Loan

File: src/model/Loan.java. Its reference fields connect the existing Member and BookCopy objects.

```
package model;

public class Loan {
    private final Member member;
    private final BookCopy copy;
    private final String startedOn;
    private final String dueOn;
    private final String status;

    public Loan(Member member, BookCopy copy,
                String startedOn, String dueOn) {
        this.member = member;
        this.copy = copy;
        this.startedOn = startedOn;
        this.dueOn = dueOn;
        this.status = "ACTIVE";
    }

    public Member getMember() {
        return member;
    }

    public BookCopy getCopy() {
        return copy;
    }

    public String getStartedOn() {
        return startedOn;
    }

    public String getDueOn() {
        return dueOn;
    }

    public String getStatus() {
        return status;
    }
}
```

10. Complete Java code: LibraryDesk

File: src/app/LibraryDesk.java. A null result means the borrowing request failed.

```
package app;

import model.Member;
import model.BookCopy;
import model.Loan;

public class LibraryDesk {

    public Loan borrow(Member member, BookCopy copy,
                       String startedOn, String dueOn) {
        // 1. Check the one-current-loan demo rule.
        if (member.hasLoan()) {
            return null;
        }

        // 2. Let the copy enforce availability.
        boolean accepted = copy.tryBorrow();
        if (!accepted) {
            return null;
        }

        // 3. Create the successful borrowing record.
        Loan loan = new Loan(
            member, copy, startedOn, dueOn
        );

        // 4. Let the member remember the record.
        member.setLoan(loan);

        // 5. Return the successful result.
        return loan;
    }
}
```

10. Complete Java code: Main, part 1

File: src/app/Main.java. Save both parts in the same Main.java file. Part 1 starts main() and demonstrates success. Continue inside main() on the next page.

```
package app;

import model.Member;
import model.Book;
import model.BookCopy;
import model.Loan;

public class Main {

    public static void main(String[] args) {
        Member member1 = new Member("M01");
        Member member2 = new Member("M02");
        Book book = new Book("Java Basics");
        BookCopy copy = new BookCopy("COPY-01", book);
        LibraryDesk desk = new LibraryDesk();

        // Scenario 1: successful borrowing.
        showState("BEFORE borrowing", member1, copy);
        Loan result = desk.borrow(
            member1, copy, "2026-10-03", "2026-10-10"
        );

        if (result != null) {
            System.out.println("Request: SUCCESS");
            System.out.println(
                "Borrower: " + result.getMember().getId()
            );
            System.out.println(
                "Book: " + result.getCopy().getBook().getTitle()
            );
            System.out.println("Due date: " + result.getDueOn());
        } else {
            System.out.println("Request: REJECTED");
        }
        showState("AFTER borrowing", member1, copy);
    }
}
```

10. Complete Java code: Main, part 2

Continuation of src/app/Main.java. Finish main(), then add showState() and the final class-closing brace.

```
// Scenario 2: another member requests this copy.
Loan secondResult = desk.borrow(
    member2, copy, "2026-10-03", "2026-10-10"
);
System.out.println();
System.out.println(
    "Second member rejected: " + (secondResult == null)
);
showState("AFTER rejected request", member2, copy);

// Scenario 3: member1 already has a current loan.
BookCopy anotherCopy = new BookCopy("COPY-02", book);
Loan thirdResult = desk.borrow(
    member1, anotherCopy, "2026-10-03", "2026-10-10"
);
System.out.println();
System.out.println(
    "Existing borrower rejected: " + (thirdResult == null)
);
System.out.println(
    "COPY-02 remains available: " + anotherCopy.isAvailable()
);
}

private static void showState(String heading,
                              Member member,
                              BookCopy copy) {
    System.out.println();
    System.out.println(heading);
    System.out.println("Member: " + member.getId());
    System.out.println("Copy: " + copy.getCopyId());
    System.out.println("Available: " + copy.isAvailable());

    Loan current = member.getLoan();
    if (current == null) {
        System.out.println("Current loan: null");
    } else {
        System.out.println("Current loan: " + current.getStatus());
    }
}
}
```